Day 1: Python Foundations & DSA

Roadmap	AI Automation Engineering + Government Technical Career
Study Date	7 November 2026
Learning Duration	2 hours
Level	Beginner-friendly foundations

Day 1 Outcome: You installed Python, configured Visual Studio Code, ran your first Python program, practised basic output and comments, and learned why data structures and algorithm efficiency matter.

Learning Objectives

- Understand what Python is and where it is used.
- Know the basic role of Python, an editor, and the interpreter.
- Run a simple Python program from VS Code.
- Use `print()` to display output and `#` to write comments.
- Understand what a data structure and an algorithm are.
- Recognise the basic meaning of O(1), O(n) and $O(n^2)$.

1. Introduction to Python

Python is a high-level, general-purpose programming language known for readable syntax and a large ecosystem of libraries. It is used in web development, data analysis, automation, artificial intelligence, machine learning, scripting, and software development.

Why Python is useful:

- **Readable syntax:** programs are often shorter and easier to understand.
- **Beginner-friendly:** you can focus on problem-solving before complex syntax.
- **Large ecosystem:** libraries support data science, AI, web applications, and automation.
- **Cross-platform:** Python is available on Windows, macOS, and Linux.

Example: Python can automate repetitive file operations, process spreadsheet data, call web APIs, or connect an application to an AI model.

2. Tools Required to Write Python

Python interpreter: software that runs Python code.
Code editor: an application in which you write and manage code.
VS Code: a popular editor that can be configured for Python development.

Basic workflow: Write code in a file ending with `.py` → run the file using Python → read the output or error → correct and run again.
Example file name: `hello.py`

3. Your First Python Program & Displaying Output

The `print()` function displays information in the output or terminal.

```
print("Hello, World!")

# Displaying multiple values:
print("My name is Suraj")
print("I am learning Python")
print(10)
print("Age:", 24)
```

How it works: `print` is a built-in Python function. The text inside quotation marks is a string. Parentheses contain the value or values passed to the function. Python executes the statement and displays the text.

4. Comments in Python

A comment is explanatory text in source code. Python ignores comments when executing the program. A single-line comment begins with `#`.

```
# This is a comment
print("Welcome to Python") # Display a message
```

Good practice: write comments to explain *why* something is done when the code is not self-explanatory. Avoid comments that merely repeat obvious code.

5. Introduction to DSA

DSA stands for Data Structures and Algorithms.

- **Data structure:** a way of organising and storing data so it can be accessed or modified efficiently. Examples include arrays/lists, stacks, queues, linked lists, trees, and graphs.
- **Algorithm:** a finite sequence of clear steps used to solve a problem or complete a task. A recipe is a useful analogy: it gives ordered instructions to produce a result.

Example: Finding a name in a list

1. Start at the first name.
2. Compare it with the name you are looking for.
3. If it matches, stop.
4. Otherwise, move to the next name.
5. If the list ends, report that the name was not found.

6. Time Complexity and Big-O Notation

Time complexity describes how an algorithm's running work grows as the input size (usually written as n) grows. Big-O notation describes an upper-bound growth category and helps compare algorithms. It is not an exact measurement of seconds.

Complexity	Common Name	Intuition
O(1)	Constant	Work stays roughly the same as input grows.
O(log n)	Logarithmic	Work grows slowly; often the input is repeatedly halved.
O(n)	Linear	Work grows in proportion to the number of items.
$O(n^2)$	Quadratic	Work can grow roughly with n multiplied by n.

Examples:
O(1) constant work: accessing an item by its index in a Python list is typically constant time (e.g., `items[0]`).
O(n) linear work: checking each item once in a list may require n checks.
$O(n^2)$ quadratic work: a nested loop that compares every item with every item can perform about n × n operations.
O(log n) logarithmic work: binary search on a sorted collection repeatedly halves the search range.

Important: Big-O focuses on growth with input size. Constants and smaller terms are usually ignored. For example, O(3n) is simplified to O(n).

7. Practice Exercises & Revision Questions

Exercises:

1. Create a file named `intro.py` and print Hello, Python!.
2. Print your name, course, and one reason you want to learn Python on separate lines.
3. Print the result of 25 + 15.
4. Add a comment at the top of your program explaining what it does.
5. Write an algorithm in plain English for finding the largest number in a list.

Quick Revision Questions:

1. What is Python, and mention two areas where it is used.
2. What is the role of the Python interpreter?
3. What does `print()` do?
4. Which symbol starts a single-line comment in Python?
5. What is the difference between a data structure and an algorithm?
6. If a loop visits each of n items once, what is its typical time complexity?
7. What is the typical complexity of two nested loops that each run n times?
8. What does O(1) mean?

8. Key Takeaways & Completion

- Python is a readable, general-purpose language used in automation, AI, data, and software development.
- A `.py` file contains Python source code; the interpreter executes it.
- Use `print()` to display output and `#` to begin a single-line comment.
- A data structure organises data; an algorithm describes steps to solve a problem.
- Big-O notation helps describe how an algorithm's work grows as input size increases.

Setup	Python installed and VS Code configured
Practical	hello.py executed successfully
Assignments	5 assignments completed
Assessment	5/5
Difficulty Reported	Easy

Study tip: Re-type the examples yourself, change the values, predict the output before running the program, and explain each line in your own words.

Next lesson preview: Python variables, naming conventions, comments in more depth, and continued DSA complexity practice.